

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Modelling Comparative Analysis Assignment

COURSE NAME: ARTIFICIAL
INTELLIGENCE
AND
EXPERT SYSTEMS

COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Modelling Comparative Analysis Assignment

Code of Conduct:

I Namburi Sai Yashaswini(192425250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Yashaswini

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Question Selected (Annexure A, Q1): Automobile Fault Diagnosis

An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. The problem is modelled below using production rules, propositional logic, first-order logic, and Prolog, with forward and backward chaining demonstrated and the approaches compared on expressiveness, inference, complexity, and suitability.

1. Problem Statement & Objectives

Domain: Automotive service and repair diagnostics at a vehicle service center.

Problem: Technicians receive vehicles exhibiting one or more observable symptoms — engine overheating, starting failure, abnormal noise, low mileage (poor fuel efficiency), and dashboard warning indicators. Manually correlating these symptoms with the underlying fault (cooling system, battery/electrical, starter motor, fuel system, ignition system, or engine mechanical failure) is slow and inconsistent across technicians. An expert system is required to encode this diagnostic knowledge and automatically infer the most probable fault and recommended corrective action from the symptoms reported.

Objectives of the modelling exercise:

- Identify and organize the domain knowledge (symptoms, faults, components, recommended actions) needed for diagnosis.
- Represent this knowledge using four modelling paradigms — production rules, propositional logic, first-order logic, and Prolog.
- Implement a working Prolog program and demonstrate both forward chaining (symptom-driven) and backward chaining (goal-driven) reasoning.
- Compare the four paradigms on expressiveness, inference mechanism, complexity, scalability, and suitability.
- Recommend the modelling approach best suited for deployment in the service center's diagnostic system.

2. Domain Knowledge & Facts

2.1 Entities

- Vehicle — the automobile being diagnosed (e.g., car1, car2, car3).
- Symptom — an observable condition reported by the driver or detected by the technician.
- Component — a physical subsystem of the vehicle (cooling system, battery, starter motor, fuel system, ignition system, exhaust/engine).
- Fault — the diagnosed root cause of the reported symptoms.
- RecommendedAction — the repair/service action associated with a fault.

2.2 Symptoms (Facts)

ID	Symptom (Fact)
S1	engine_overheating
S2	coolant_level_low
S3	radiator_blocked
S4	starting_failure
S5	battery_weak
S6	dashboard_warning_light
S7	clicking_sound_on_start
S8	abnormal_noise
S9	knocking_sound
S10	low_mileage
S11	fuel_smell
S12	exhaust_smoke
S13	engine_misfire
S14	spark_plug_worn

2.3 Faults (Conclusions)

ID	Fault (Conclusion)
F1	cooling_system_failure
F2	battery_failure
F3	starter_motor_failure
F4	fuel_system_failure
F5	ignition_system_failure
F6	engine_mechanical_failure

2.4 Relationships & Conditions

- Each fault is concluded from a conjunction of two or more co-occurring symptoms (e.g., overheating together with low coolant, rather than either symptom alone).
- A single symptom (e.g., abnormal_noise) is ambiguous by itself; it must be combined with a distinguishing symptom (e.g., knocking_sound) to isolate the correct fault.
- Some conditions require the explicit absence of a symptom (negation) to disambiguate two faults that share a symptom — e.g., starting_failure appears in both battery_failure and starter_motor_failure, so battery_weak is used to tell them apart.
- Every fault maps to exactly one recommended action, forming a Fault → RecommendedAction relationship used after diagnosis.

3. Production Rule Model

The domain knowledge is represented as IF–THEN production rules operating on the symptom facts defined in Section 2.

R1: IF engine_overheating AND coolant_level_low THEN cooling_system_failure

R2: IF engine_overheating AND radiator_blocked THEN cooling_system_failure

R3: IF starting_failure AND battery_weak AND dashboard_warning_light THEN battery_failure

R4: IF starting_failure AND clicking_sound_on_start AND NOT battery_weak THEN
starter_motor_failure

R5: IF low_mileage AND fuel_smell THEN fuel_system_failure

R6: IF exhaust_smoke AND low_mileage THEN fuel_system_failure

R7: IF engine_misfire AND spark_plug_worn THEN ignition_system_failure

R8: IF abnormal_noise AND knocking_sound THEN engine_mechanical_failure

R9: IF cooling_system_failure THEN recommend(check_coolant_and_service_radiator)

R10: IF battery_failure THEN recommend(charge_or_replace_battery)

How conclusions are derived: rules R1–R8 map raw symptom facts to intermediate fault conclusions, while R9–R10 chain off those fault conclusions to derive a recommended action (rule chaining). In forward chaining, the system starts from the reported symptoms and fires any rule whose full antecedent is satisfied, adding the consequent as a new fact; this repeats until a fault (and its recommendation) is asserted. In backward chaining, the system starts from a candidate fault goal and works backward, checking whether the required symptom facts hold, recursively, until the goal is proved or disproved.

4. Propositional Logic Model

Each symptom and fault is represented as an atomic proposition; the production rules are re-expressed using logical connectives ($\wedge$ = AND, $\vee$ = OR, $\neg$ = NOT, $\rightarrow$ = IMPLIES).

4.1 Propositions

Symbol	Meaning
P1	engine_overheating
P2	coolant_level_low
P3	radiator_blocked
P4	starting_failure
P5	battery_weak
P6	dashboard_warning_light
P7	clicking_sound_on_start
P8	abnormal_noise
P9	knocking_sound
P10	low_mileage
P11	fuel_smell

P12	exhaust_smoke
P13	engine_misfire
P14	spark_plug_worn

Symbol	Meaning
Q1	cooling_system_failure
Q2	battery_failure
Q3	starter_motor_failure
Q4	fuel_system_failure
Q5	ignition_system_failure
Q6	engine_mechanical_failure

4.2 Logical Formulas

$(P1 \wedge P2) \rightarrow Q1$

$(P1 \wedge P3) \rightarrow Q1$

$(P4 \wedge P5 \wedge P6) \rightarrow Q2$

$(P4 \wedge P7 \wedge \neg P5) \rightarrow Q3$

$(P10 \wedge P11) \rightarrow Q4$

$(P12 \wedge P10) \rightarrow Q4$

$(P13 \wedge P14) \rightarrow Q5$

$(P8 \wedge P9) \rightarrow Q6$

4.3 Inference Statement (Modus Ponens Example)

Given facts P1 (engine_overheating) is true and P2 (coolant_level_low) is true, and the formula $(P1 \wedge P2) \rightarrow Q1$, Modus Ponens allows us to infer Q1 (cooling_system_failure) is true. Propositional logic can validate this single, fixed instance, but since P1–P14 are flat statements with no vehicle argument, a separate copy of every formula would be required for every vehicle in the service center — it cannot generalize the rule 'for any vehicle' the way predicate logic can.

5. First-Order Logic Model

First-order logic (FOL) generalizes the propositional model by introducing variables, predicates, and quantifiers, so a single rule applies to any vehicle.

5.1 Predicates & Constants

- $\text{HasSymptom}(v, s)$ — vehicle v exhibits symptom s .
- $\text{Fault}(v, f)$ — vehicle v is diagnosed with fault f .
- $\text{Recommends}(f, a)$ — fault f is associated with recommended action a .
- $\text{ShouldPerform}(v, a)$ — action a should be performed on vehicle v .
- Constants: car1 , car2 , car3 (vehicles); $\text{engine_overheating}$, coolant_level_low , ... (symptom constants); $\text{cooling_system_failure}$, battery_failure , ... (fault constants).

5.2 Quantified FOL Rules

$\forall v [(\text{HasSymptom}(v, \text{engine_overheating}) \wedge \text{HasSymptom}(v, \text{coolant_level_low})) \rightarrow \text{Fault}(v, \text{cooling_system_failure})]$

$\forall v [(\text{HasSymptom}(v, \text{engine_overheating}) \wedge \text{HasSymptom}(v, \text{radiator_blocked})) \rightarrow \text{Fault}(v, \text{cooling_system_failure})]$

$\forall v [(\text{HasSymptom}(v, \text{starting_failure}) \wedge \text{HasSymptom}(v, \text{battery_weak}) \wedge \text{HasSymptom}(v, \text{dashboard_warning_light})) \rightarrow \text{Fault}(v, \text{battery_failure})]$

$\forall v [(\text{HasSymptom}(v, \text{starting_failure}) \wedge \text{HasSymptom}(v, \text{clicking_sound_on_start}) \wedge \neg \text{HasSymptom}(v, \text{battery_weak})) \rightarrow \text{Fault}(v, \text{starter_motor_failure})]$

$\forall v [(\text{HasSymptom}(v, \text{low_mileage}) \wedge \text{HasSymptom}(v, \text{fuel_smell})) \rightarrow \text{Fault}(v, \text{fuel_system_failure})]$

$\forall v [(\text{HasSymptom}(v, \text{engine_misfire}) \wedge \text{HasSymptom}(v, \text{spark_plug_worn})) \rightarrow \text{Fault}(v, \text{ignition_system_failure})]$

$\forall v [(\text{HasSymptom}(v, \text{abnormal_noise}) \wedge \text{HasSymptom}(v, \text{knocking_sound})) \rightarrow \text{Fault}(v, \text{engine_mechanical_failure})]$

$\forall v \forall f \forall a [(\text{Fault}(v, f) \wedge \text{Recommends}(f, a)) \rightarrow \text{ShouldPerform}(v, a)]$

Existential example: $\exists v \text{Fault}(v, \text{battery_failure})$ — 'there exists a vehicle currently diagnosed with battery failure', used e.g. to query whether any vehicle in the service bay needs battery attention today.

6. Prolog Model & Implementation

The FOL rules above are directly encoded as Prolog Horn clauses (facts and rules).

% ---- Facts: symptoms observed per vehicle ----

symptom(car1, engine_overheating).

symptom(car1, coolant_level_low).

symptom(car2, starting_failure).

symptom(car2, battery_weak).

symptom(car2, dashboard_warning_light).

symptom(car3, low_mileage).

symptom(car3, fuel_smell).

symptom(car4, starting_failure).

symptom(car4, clicking_sound_on_start).

% ---- Rules: symptom -> fault ----

fault(V, cooling_system_failure) :-

 symptom(V, engine_overheating),

 symptom(V, coolant_level_low).

fault(V, cooling_system_failure) :-

 symptom(V, engine_overheating),

 symptom(V, radiator_blocked).

fault(V, battery_failure) :-

 symptom(V, starting_failure),

 symptom(V, battery_weak),

 symptom(V, dashboard_warning_light).

fault(V, starter_motor_failure) :-

 symptom(V, starting_failure),

 symptom(V, clicking_sound_on_start),

 \+ symptom(V, battery_weak).

fault(V, fuel_system_failure) :-

 symptom(V, low_mileage),

```
symptom(V, fuel_smell).
```

```
fault(V, ignition_system_failure) :-
```

```
    symptom(V, engine_misfire),
```

```
    symptom(V, spark_plug_worn).
```

```
fault(V, engine_mechanical_failure) :-
```

```
    symptom(V, abnormal_noise),
```

```
    symptom(V, knocking_sound).
```

```
% ---- Fault -> recommended action ----
```

```
recommend(cooling_system_failure, 'Check coolant level and service radiator').
```

```
recommend(battery_failure, 'Charge or replace battery').
```

```
recommend(starter_motor_failure, 'Inspect and replace starter motor').
```

```
recommend(fuel_system_failure, 'Inspect fuel system for leaks/clogs').
```

```
recommend(ignition_system_failure, 'Replace spark plugs, check ignition timing').
```

```
recommend(engine_mechanical_failure, 'Inspect engine internals for wear/damage').
```

```
diagnose(V, Fault, Action) :-
```

```
    fault(V, Fault),
```

```
    recommend(Fault, Action).
```

6.1 Sample Queries & Output

```
?- fault(car1, F).
```

```
F = cooling_system_failure.
```

```
?- diagnose(car2, Fault, Action).
```

```
Fault = battery_failure,
```

```
Action = 'Charge or replace battery'.
```

```
?- fault(car3, F).
```

```
F = fuel_system_failure.
```

```
?- diagnose(car4, Fault, Action).  
Fault = starter_motor_failure,  
Action = 'Inspect and replace starter motor'.
```

7. Forward and Backward Chaining Demonstration

7.1 Forward Chaining (data-driven) — car1

Working memory (initial facts): engine_overheating(car1), coolant_level_low(car1).

- Step 1: Scan rule base for a rule whose antecedent is fully satisfied by working memory. R1 requires engine_overheating $\wedge$ coolant_level_low — both present.
- Step 2: Fire R1 $\rightarrow$ assert new fact cooling_system_failure(car1) into working memory.
- Step 3: Re-scan rule base with the updated working memory. R9 requires cooling_system_failure — now present.
- Step 4: Fire R9 $\rightarrow$ assert recommend(car1, check_coolant_and_service_radiator).
- Step 5: No further rule antecedents are satisfied — halt. Result: car1 $\rightarrow$ cooling_system_failure $\rightarrow$ check coolant / service radiator.

7.2 Backward Chaining (goal-driven) — car2, goal: fault(car2, battery_failure)

- Step 1: Goal — prove fault(car2, battery_failure).
- Step 2: Match goal against rule R3's consequent; adopt R3's antecedent as the new subgoal list: starting_failure(car2), battery_weak(car2), dashboard_warning_light(car2).
- Step 3: Check subgoal starting_failure(car2) against facts — true.
- Step 4: Check subgoal battery_weak(car2) against facts — true.
- Step 5: Check subgoal dashboard_warning_light(car2) against facts — true.
- Step 6: All subgoals proven $\rightarrow$ the original goal fault(car2, battery_failure) succeeds.

Contrast: forward chaining is data-driven and well suited to a technician entering all observed symptoms up front (as in car1); backward chaining is goal-driven and well suited to verifying/ruling out one suspected fault at a time (as in car2), which is closer to how Prolog itself executes queries.

8. Comparative Analysis of Modelling Approaches

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
-----------	------------------	---------------------	-------------------	--------

Knowledge Representation	IF–THEN rules	Atomic propositions & connectives	Predicates, variables, quantifiers	Facts & Horn-clause rules
Expressiveness	Moderate	Low – no variables/relations	High – variables & relations	High – executable FOL subset
Rule Representation	Direct IF–THEN	Implication formulas	Quantified implications	Head :- Body clauses
Inference Mechanism	Pattern/rule matching	Truth tables / resolution	Unification + resolution	SLD resolution (built-in)
Forward Chaining	Native, data-driven	Manual, no built-in engine	Possible, needs custom engine	Not native; can be simulated
Backward Chaining	Supported, goal-driven	Manual, limited	Supported via refutation	Native execution model
Handling Relationships	Weak – flat conditions	None	Strong – relations between objects	Strong – relational facts
Scalability	Moderate – rule growth	Low – combinatorial explosion	Moderate – expressive but heavy	High – efficient indexing
Explainability	High – clear rule trace	Moderate	Moderate – formal, less intuitive	High – query trace
Ease of Implementation	Easy – simple rule engine	Easy but limited use	Difficult – custom engine needed	Easy – built-in interpreter
Suitability for the Problem	Good fit	Poor – can't generalize	Good conceptually, heavy to build	Best – power + practicality

9. Results, Discussion and Model Selection

Applying all four models to the same six-fault automobile diagnosis problem confirms the expected trade-off: propositional logic is the easiest to reason about but cannot generalize across vehicles without duplicating every rule per vehicle instance; first-order logic fixes this with variables and quantifiers but has no efficient built-in execution engine of its own; production rules are intuitive and map directly onto how a technician thinks (symptom $\rightarrow$ fault $\rightarrow$ action) and support both forward and backward chaining naturally; Prolog operationalizes first-order Horn-clause logic with an efficient, built-in SLD-resolution engine, giving it FOL's expressiveness together with production rules' ease of execution and explainability.

Recommended model: Prolog, used together with a production-rule framing of the knowledge base. Prolog gives the service center a working, queryable diagnostic engine (Section 6) while the production-rule layer (Section 3) keeps the knowledge base easy for non-programmer technicians to read, audit, and extend.

Limitations:

- The knowledge base is hand-encoded; adding a new fault or symptom requires a developer to write new rules/clauses.
- No handling of uncertainty — a symptom is treated as strictly true/false, whereas real diagnoses often involve partial or conflicting evidence.
- No learning from past repair outcomes; the system cannot improve its rules automatically over time.

Possible improvements:

- Introduce certainty factors or fuzzy membership values on symptoms and rules to express diagnostic confidence.
- Combine the rule base with a machine-learning classifier trained on historical repair records for symptoms that are noisy or continuous (e.g., sensor readings).
- Add an explanation module that renders the forward/backward chaining trace to the technician in natural language.

10. Conclusion, References and GitHub Repository

This assignment modelled the automobile fault-diagnosis problem across four knowledge-representation paradigms, demonstrated forward and backward chaining, and compared the paradigms on expressiveness, inference, scalability, and explainability. Prolog, layered over a production-rule-

style knowledge base, was identified as the most suitable approach for a deployable service-center diagnostic system, balancing formal expressiveness with practical, explainable execution.

References:

- Russell, S. and Norvig, P. — Artificial Intelligence: A Modern Approach (knowledge representation, logic, and inference chapters).
- Bratko, I. — Prolog Programming for Artificial Intelligence.
- Giarratano, J. and Riley, G. — Expert Systems: Principles and Programming (production rules, forward/backward chaining).
- SWI-Prolog official documentation, <https://www.swi-prolog.org/>

GitHub Repository:

<https://github.com/yashaswini/automobile-fault-diagnosis-expert-system> (replace with your actual repository URL before submission; it should contain the Prolog source file, this report, and the test queries/outputs from Section 6.1).